CS7CS4/CSU44061 Machine Learning
Student Name: Stephen Komolafe
Student Number: 21336975

Dataset ID: # id:14-14--14

Week 3 Assignment – Lasso & Ridge regression

Part i:

(a) Figure 1 presents a 3D visualisation of the dataset as a scatter plot, where each point represents one training observation. The x-axis and y-axis correspond to the two input features (X_1 and X_2), while the z-axis represents the real-valued target variable (y). The data was extracted using the first two columns of the dataset as features and the third column as the target variable:

```
X = file.iloc[:, 0:2].values
y = file.iloc[:, 2].values
```

The transparency of the points reflects their depth in the 3D space, with more opaque points appearing in the foreground, while transparent ones recede into the background. The plot can be rotated interactively within the development environment to examine the data distribution from different angles.

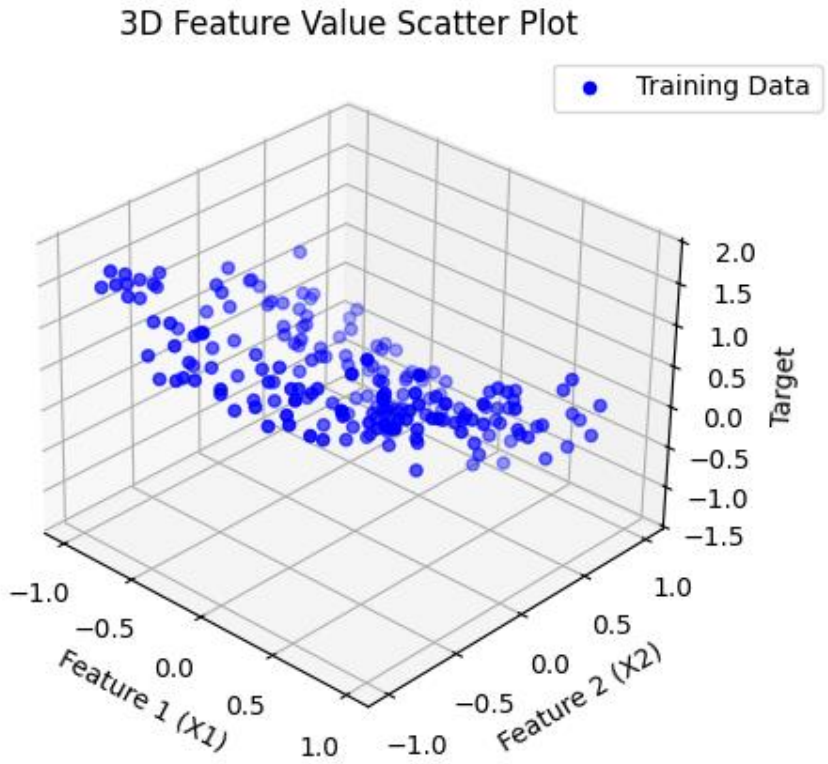

Figure 1. Visualisation of Dataset

By viewing the plot from multiple perspectives, it becomes apparent that the data forms a curved surface rather than a flat plane, indicating a nonlinear relationship between the input features and the target variable.

(b) To investigate how regularisation affects model complexity, Lasso regression models were trained using the `Lasso()` function from `sklearn.linear_model`. The models used the two original features (X_1 and X_2) along with all polynomial combinations of these features up to the 5th degree, generated using the `PolynomialFeatures()` function from `sklearn.preprocessing`. The models were trained for a range of regularisation strengths corresponding to C=[0.1, 1, 10, 100, 1000].

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Lasso

poly = PolynomialFeatures(degree=5, include_bias=False)
X_poly = poly.fit_transform(X)
feature_names = poly.get_feature_names_out(['x1', 'x2'])

C_values = [0.1, 1, 10, 100, 1000]
alphas = [1 / (2 * C) for C in C_values]
results_lasso = []

    for i, (C, alpha) in enumerate(zip(C_values, alphas)):
        model = Lasso(alpha=alpha, max_iter=10000)
        model.fit(X_poly, y)
        results_lasso.append({'C': C,
                              'Intercept': model.intercept_,
                              **dict(zip(feature_names, model.coef_))
                              })
```

The Lasso regression model takes the general form:

$$\hat{y} = \theta_0 + \sum_{j=1}^{n} \theta_j \left(poly_j\right)$$

where θ_0 is the intercept and each $poly_j$ term represents one of the polynomial feature combinations up to x_1^5 and x_2^5:

$$\sum_{j=1}^{n} \theta_j \left(poly_j\right) = (\theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1 x_2 + \theta_5 x_2^2 + \theta_6 x_1^3 + \theta_7 x_1^2 x_2 + \theta_8 x_1 x_2^2 + \theta_9 x_2^3 + \theta_{10} x_1^4 + \theta_{11} x_1^3 x_2 + \theta_{12} x_1^2 x_2^2 + \theta_{13} x_1 x_2^3 + \theta_{14} x_2^4 + \theta_{15} x_1^5 + \theta_{16} x_1^4 x_2 + \theta_{17} x_1^3 x_2^2 + \theta_{18} x_1^2 x_2^3 + \theta_{19} x_1 x_2^4 + \theta_{20} x_2^5)$$

The resulting model coefficients are summarised in Table 1 below:

C	(Intercept) θ_0	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7	θ_8	θ_9	θ_{10}	θ_{11}	θ_{12}	θ_{13}	θ_{14}	θ_{15}	θ_{16}	θ_{17}	θ_{18}	θ_{19}	θ_{20}
0.1	0.3524	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0.3524	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10	0.242	0	-0.8166	0.2919	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
100	0.06	0	-0.9382	0.8149	0	0	0	0	0	-0.0191	0	0	0	0	0	0	0	0	0	0	0
1000	0.0151	-0.0285	-0.8975	0.9751	-0.0889	0.202	0	0	0.1863	-0.0962	-0.1341	0.1204	0	0	-0.2729	0.0238	0.0514	0.0191	-0.1307	-0.2634	0

Table 1 - Lasso Model Parameters (θ) per Penalty Parameter (C)

The cost function for Lasso regression includes an L_1 penalty term ($|\theta_j|$), which encourages sparsity in the model by shrinking the coefficients of features that contribute little to the prediction task exactly to zero:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \frac{1}{C}\sum_{j=1}^{n} |\theta_j|$$

For small C values (C=0.1, 1), the strong regularisation suppressed the influence of both features, causing all coefficients to shrink to zero and leaving only the intercept term. This resulted in a constant prediction and clear underfitting.

@$C = 0.1$ & 1:	$\hat{y} = 0.3524$

As C increased, the L_1 penalty weakened, and more coefficients became non-zero. At C=10, only the coefficients for x_2 and x_1^2 were retained, while at C=100 and 1000, the coefficients of higher-order and interaction terms (i.e. x_1x_2) emerged. The intercept also shrinks towards zero as the number of non-zero coefficients increase.

@ $C = 10$:	$\hat{y} = 0.242 - 0.8166x_2 + 0.2919x_1^2$
@ $C = 100$:	$\hat{y} = 0.06 - 0.9382x_2 + 0.8149x_1^2 - 0.0191x_2^3$
@ $C = 1000$:	$\hat{y} = 0.0151 - 0.0285x_1 - 0.8975x_2 + 0.9751x_1^2 - 0.0889x_1x_2 + 0.202x_2^2 + 0.1863x_1x_2^2 - 0.0962x_2^3 - 0.1341x_1^4 + 0.1204x_1^3x_2 - 0.2729x_2^4 + 0.0238x_1^5 + 0.0514x_1^4x_2 + 0.0191x_1^3x_2^2 - 0.1307x_1^2x_2^3 - 0.2634x_1x_2^4$

By gradually increasing C, the model captured increasingly complex relationships but also became more prone to overfitting. This demonstrates Lasso's property of enforcing sparsity through feature selection by driving less important features exactly to zero.

(c) For each value of the regularisation parameter C= [0.1, 1, 10, 100, 1000], a corresponding Lasso regression model was fitted to the training data, and prediction surfaces were generated to visualise how the fitted model changes with regularisation strength. A grid of feature values (x_1,x_2) was created using the `numpy.meshgrid()` function, extending slightly beyond the range of the original dataset:

```
x1_range = np.linspace(-1.5, 1.5, 50)
x2_range = np.linspace(-1.5, 1.5, 50)
x1_grid, x2_grid = np.meshgrid(x1_range, x2_range)
Xtest = np.c_[x1_grid.ravel(), x2_grid.ravel()]
```

This grid was transformed using the same 5th-degree polynomial feature mapping applied during training, and predictions were generated for each grid point using the fitted models:

```
Xtest_poly = poly.transform(Xtest)
y_pred = model.predict(Xtest_poly)
y_pred_grid = y_pred.reshape(x1_grid.shape)
```

Figure 2 shows the resulting prediction surfaces for each regularisation strength C, plotted alongside the original training data to demonstrate how model complexity evolves as the regularisation weakens.

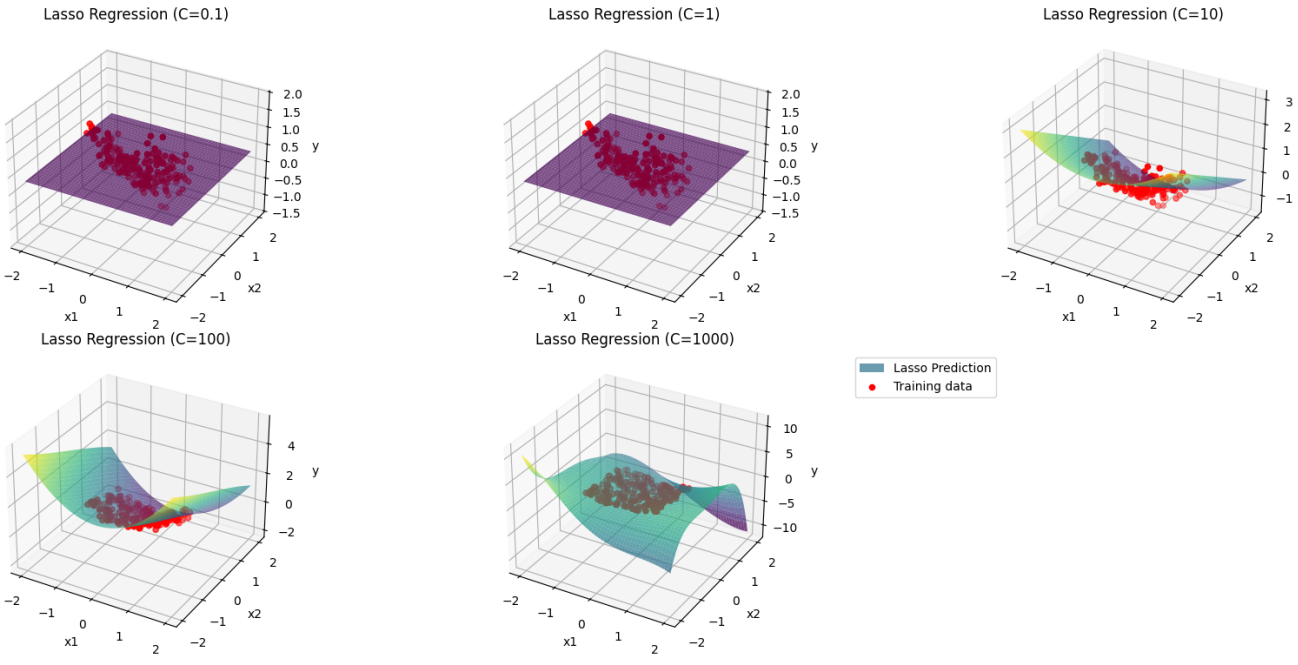

Figure 2. Lasso Regression Predictions versus Training Data per C

For small C values (C=0.1, 1), the L_1 penalty is strong, driving all coefficients to zero and leaving only the intercept term (θ_0). As a result, the predicted surface is completely flat, representing a constant prediction across all input values.

As C increases to 10, the surface begins to curve, reflecting simple nonlinear relationships mainly driven by x_2 and x_1^2. The model starts to follow the general trend of the data, though remaining relatively simple.

Once C reaches 100, the curvature of the surface becomes more detailed, capturing additional nonlinear structure as more coefficients remain non-zero. However, when C continues to 1000, the prediction surface becomes overly complex, suggesting that the model has begun to overfit the training data.

(d) Underfitting occurs when a model is too simple to capture underlying relationships in the data, resulting in high error on both the training and test sets. Overfitting refers to when a model becomes too complex and starts fitting noise or random fluctuations in the training data, leading to low training error but poor generalisation to unseen data.

In Lasso regression, the penalty weight parameter C (where the regularisation strength is defined as $\alpha = \frac{1}{2C}$) directly controls the trade-off between model complexity and regularisation strength. When this C value is small, the regularisation is strong, heavily penalising large coefficients and forcing many of them to zero. This results in a very simple model that is prone to underfitting. For instance, at C=0.1 and C=1, all coefficients except the intercept reduced to zero, producing a flat prediction surface that failed to capture any meaningful relationship between x_1, x_2, and y.

As C increases, the L_1 penalty weakens, allowing more coefficients to take non-zero values. This increases model flexibility and enables it to capture nonlinear patterns in the data. At moderate C values such as C=10 or C=100, the model begins to capture important nonlinear patterns without excessive noise fitting. However, when C becomes very large (e.g.,, C=1000), the model retains many higher-order and interaction terms, resulting in a highly curved and complex surface. This suggests the onset of overfitting, as the model begins to capture noise in the training data rather than the underlying general pattern.

(e) The Ridge regression models were trained following the same procedure as the Lasso models, using the two original features x_1 and x_2 together with all polynomial combinations of these features up to the 5th degree. The models were trained for a range of regularisation strengths corresponding to C= [0.1, 1, 10, 100, 1000], using the `Ridge()` function from `sklearn.linear_model`.

```
from sklearn.linear_model import Ridge

results_ridge = []
for idx, (C, alpha) in enumerate(zip(C_values, alphas)):
        model = Ridge(alpha=alpha, max_iter=10000)
        model.fit(X_poly, y)
        results_ridge.append({'C': C,
                              'Intercept': model.intercept_,
                              **dict(zip(feature_names, model.coef_))
                             })
```

The resulting model coefficients are summarised in Table 2 below:

C	(Intercept) θ_0	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6	θ_7	θ_8	θ_9	θ_{10}	θ_{11}	θ_{12}	θ_{13}	θ_{14}	θ_{15}	θ_{16}	θ_{17}	θ_{18}	θ_{19}	θ_{20}
0.1	0.3524	0.1139	-0.0087	-0.6936	0.4783	-0.032	0.0066	0.0038	-0.1133	0.0151	-0.2275	0.2695	0.0326	0.1019	0.0056	-0.0638	0.015	-0.0221	0.0134	-0.0427	-0.0339
1	0.3524	0.0346	-0.0271	-0.8562	0.835	-0.1427	0.1845	-0.0197	-0.057	0.1499	-0.1538	0.0109	0.1645	0.0037	0.0454	-0.2557	0.0467	0.1077	0.0728	-0.1304	-0.25
10	0.242	-0.0244	-0.0475	-0.9291	1.1003	-0.2018	0.4447	-0.1248	-0.0225	0.5777	0.0005	-0.2343	0.244	-0.1513	0.0327	-0.5268	0.1493	0.262	0.0433	-0.3594	-0.7768
100	0.06	-0.037	-0.0635	-0.9618	1.1477	-0.2035	0.519	-0.1607	-0.0063	0.7947	0.1118	-0.2732	0.2588	-0.2014	0.0082	-0.6066	0.1946	0.3022	-0.0102	-0.4392	-1.012
1000	0.0151	-0.0384	-0.0661	-0.9668	1.1527	-0.2033	0.5282	-0.1644	-0.0033	0.8255	0.1298	-0.2771	0.2605	-0.2079	0.0044	-0.6167	0.2002	0.3064	-0.0188	-0.45	-1.0446

Table 2 - Ridge Model Parameters (θ) per Penalty Parameter (C)

Unlike Lasso regression, Ridge regression includes an L_2 penalty term ($\theta_j{}^2$), which adds the squared sum of all coefficients to the cost function. This penalty distributes shrinkage smoothly across all features and does not force any coefficient exactly to zero. The Ridge cost function can be expressed as:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right)^2 + \frac{1}{C}\sum_{j=1}^{n}\left(\theta_j\right)^2$$

While C=0.1, the strong regularisation causes all coefficient magnitudes to shrink toward zero, though not exactly to zero. This shows that the model still retains all polynomial terms, albeit weakly, resulting in mild underfitting. This behaviour contrasts with Lasso regression, where many coefficients are driven exactly to zero, resulting in sparse models.

As C increases (e.g., C=1, 10, 100, 1000), the strength of the L_2 penalty decreases, letting coefficients grow gradually in magnitude. Unlike Lasso, this transition occurs smoothly, being that there is no jump from zero to non-zero coefficient values, allowing the model to continuously adjusts all coefficients rather than selecting only a few dominant terms.

For each value of the regularisation parameter C= [0.1, 1, 10, 100, 1000], the corresponding Ridge regression model was then trained and used to generate predictions over the same grid of (x_1,x_2) values as in the Lasso case.

```
for idx, (C, alpha) in enumerate(zip(C_values, alphas)):
    model = Ridge(alpha=alpha, max_iter=10000)
    model.fit(X_poly, y)
    results_ridge.append({
        'C': C,
        'Intercept': model.intercept_,
        **dict(zip(feature_names, model.coef_))
    })
    Xtest_poly = poly.transform(Xtest)
    y_pred = model.predict(Xtest_poly)
    y_pred_grid = y_pred.reshape(x1_grid.shape)
```

Figure 3 presents the resulting prediction surfaces alongside the original training data.

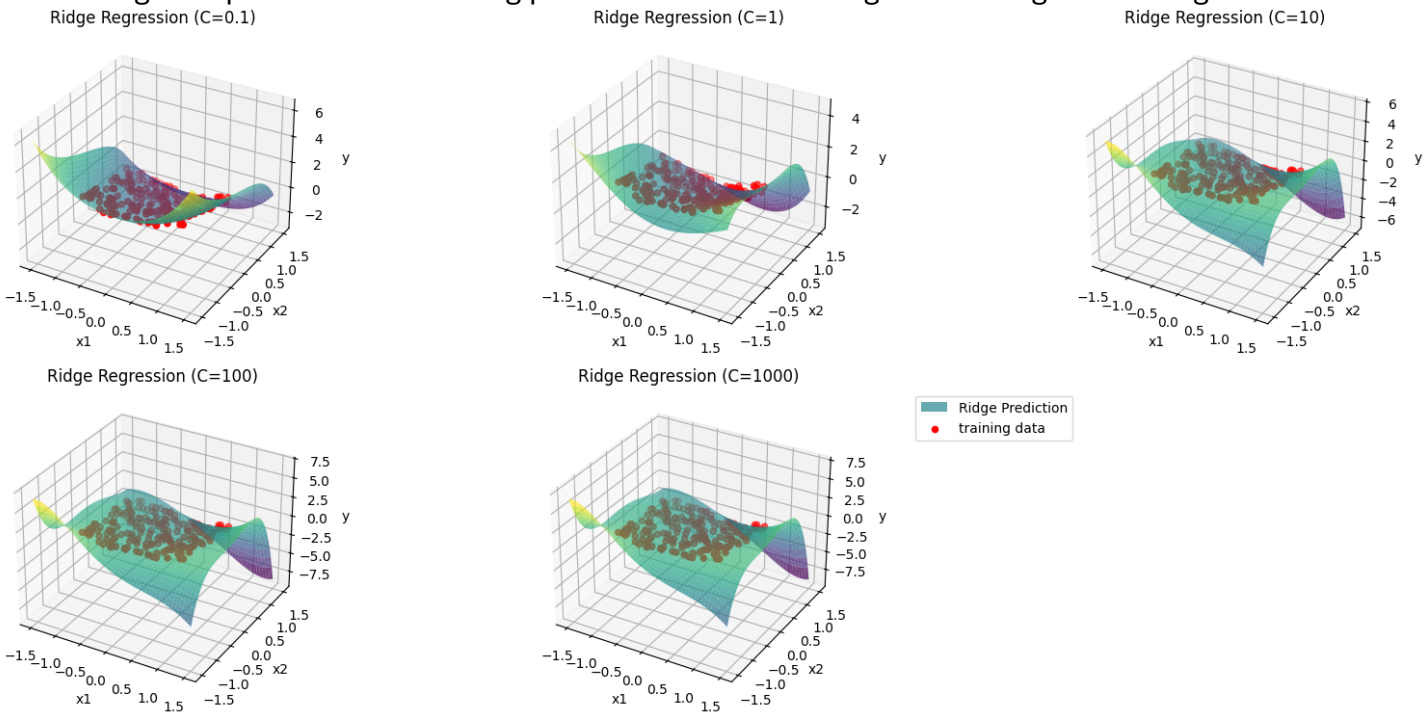

Figure 3. Ridge Regression Predictions versus Training Data per C

At C=0.1, the strong L_2 penalty heavily constrains the coefficients, producing a relatively flat and smooth surface. However, unlike Lasso, the coefficients are not reduced exactly to zero, so the surface still exhibits a subtle curvature rather than being completely flat.

As C increases (e.g., C=1, 10), the regularisation weakens, and the model begins to capture more of the data's underlying nonlinear structure. The surface gradually gains curvature while remaining smooth and continuous due to the nature of the L_2 penalty.

For larger values of C (e.g., C=100, 1000), the model becomes more flexible, producing increasingly detailed nonlinear surfaces similar to those of the Lasso model at equivalent C values. However, the change between Ridge surfaces at C=100 and C=1000 remains smoother and less erratic, as the L_2 penalty prevents large, isolated coefficient values.

From this it is evident that Ridge regression produces less sparse but smoother prediction surfaces compared to Lasso regression. It distributes influence across all polynomial terms, resulting in gradual transitions between underfitting and overfitting as C increases, rather than the abrupt changes characteristic of Lasso.

Part ii:

(a) 5-fold cross validation was conducted to evaluate the performance of Lasso regression models across a logarithmic range of regularisation strengths, with C values ranging from 0.01 up to 10000. Cross-validation scores were computed using the `cross_val_score` function from `sklearn.model_selection`, in combination with the `mean_squared_error` and `make_scorer` functions from `sklearn.metrics`.

```
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_squared_error, make_scorer

C_values = [0.01, 0.1, 1, 10, 100, 1000, 10000]
alphas = [1 / (2 * C) for C in C_values]

results_5FCV = []
mean_errors = []
std_errors = []

for C, alpha in zip(C_values, alphas):
    model = Lasso(alpha=alpha, max_iter=10000)
    scores = cross_val_score(model, X_poly, y, cv=5, scoring='neg_mean_squared_error'))
    mse_scores = -scores
    mean_errors.append(mse_scores.mean())
    std_errors.append(mse_scores.std())
    results_5FCV.append({"C": C,
                         "Alpha": alpha,
                         "Mean_MSE": mse_scores.mean(),
                         "Std_MSE": mse_scores.std()
                        })
```

For each fold, the model was trained on a subset of the data and evaluated on the remaining portion, and the mean and standard deviation of the mean squared error (MSE) were computed. The results are summarised in **Table 3** below.

C	Mean MSE	Std MSE
0.01	0.4318	0.0967
0.1	0.4318	0.0967
1	0.4318	0.0967
10	0.0899	0.0215
100	0.0533	0.0073
1000	0.0609	0.0113
10000	0.0632	0.0104

Table 3 - [Lasso] Mean Squared Errors & Standard Deviations per C

These results were visualised by plotting the mean cross-validation error against C, with standard deviations represented as error bars using the `errorbar` function.

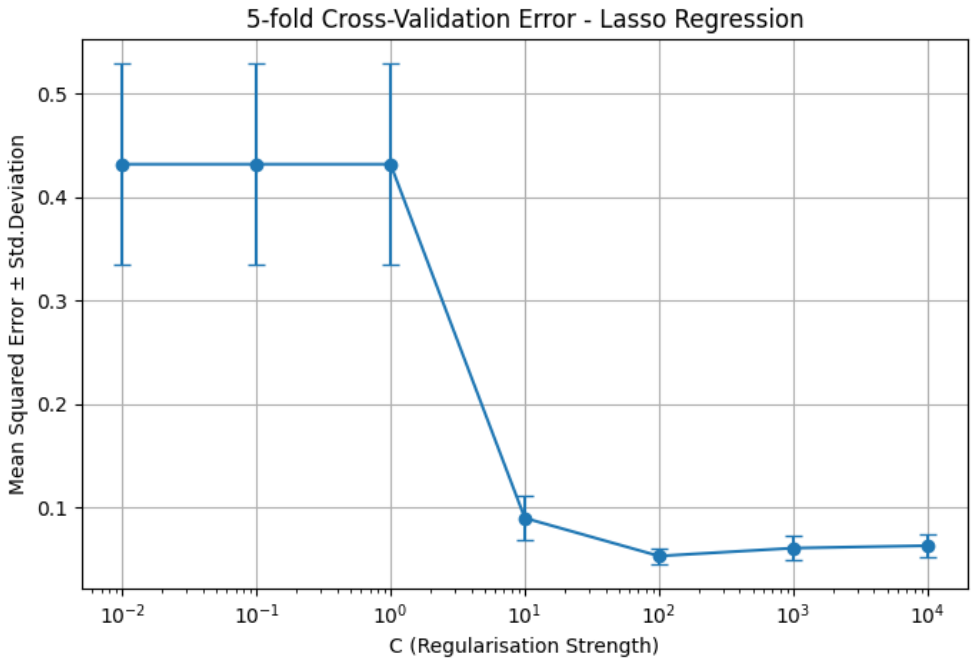

Figure 4. 5-Fold Cross-Validation Error for Lasso Regression Model

As shown in Figure 4, the selected range of C values was chosen to span both strong and weak regularisation strengths. In section (i)(c), it was observed that when C approached 1000, the prediction surface of the Lasso model began to show signs of overfitting. To investigate this behaviour further, the range was extended up to C =10000. Despite this extension, the increase in error from C=100 to C=10000 was minimal, indicating that the performance of the model had stabilised. This suggests that beyond C=100, the reduction in regularisation strength no longer yields meaningful improvement and introduces little additional overfitting.

(b) For smaller C values (0.01–10), strong L_1 regularisation in the Lasso model forced most coefficients to zero, leading to underfitting and higher prediction errors. As C increased, the regularisation weakened, reducing bias and improving the model's predictive accuracy.
Although the range of C values was extended up to 10000 to further examine potential overfitting, the increase in error beyond C=100 was minimal. Therefore, C=100 would be the recommended value of C to use, corresponding to the lowest mean cross-validation error of 0.0533 with a small standard deviation of 0.0073, showing stable model performance across folds.
However, if model simplicity and sparsity were to be prioritised over performance, C=10 could also be considered a reasonable alternative, offering a simpler but slightly less accurate model.

(c) 5-fold cross validation was performed again, but this time to evaluate Ridge regression models across the same range of regularisation strengths, from C=0.01 up to C=10000. The `cross_val_score` function from `sklearn.model_selection` was used with `mean_squared_error` as the scoring metric.

```
rdgResults_5FCV = []
rdgMean_errors = []
rdgStd_errors = []
alphas = [1 / C for C in C_values]

for C, alpha in zip(C_values, alphas):
    model = Ridge(alpha=alpha, max_iter=10000)
    scores = cross_val_score(model, X_poly, y, cv=5,
                             scoring='neg_mean_squared_error'))
    mse_scores = -scores
    rdgMean_errors.append(mse_scores.mean())
    rdgStd_errors.append(mse_scores.std())
    rdgResults_5FCV.append({"C": C,
                            "Alpha": alpha,
                            "Mean_MSE": mse_scores.mean(),
                            "Std_MSE": mse_scores.std()
                             })
```

For each fold, the model is fitted to the training data, and the mean squared errors (MSE) and standard deviations are computed as shown in table 4 below:

C	Mean MSE	Std MSE
0.01	0.1892	0.0526
0.1	0.0709	0.0179
1	0.0617	0.0142
10	0.0612	0.0116
100	0.0635	0.0103
1000	0.0642	0.0101
10000	0.0643	0.0101

Table 4 - [Ridge] Mean Squared Errors & Standard Deviations per C

These results were visualised by plotting the mean cross-validation error against C, with standard deviations represented as error bars using the `errorbar` function.

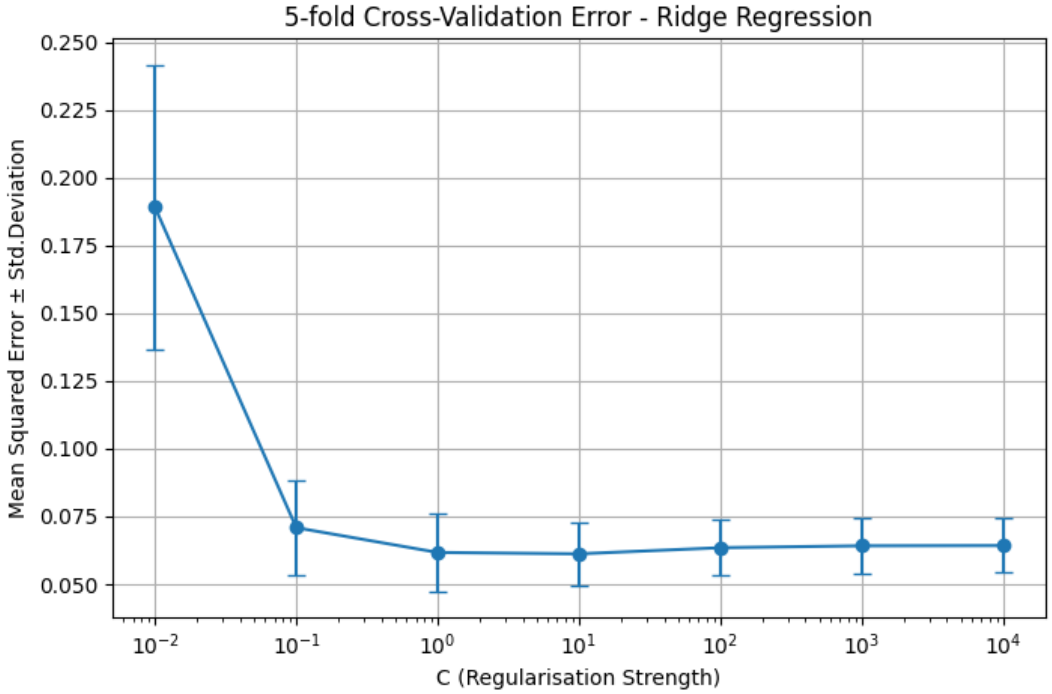

Figure 5. 5-Fold Cross-Validation Error for Lasso Regression Model

As shown in Figure 5, the Ridge model exhibited its highest mean error and variability at the smallest C value (0.01), indicating underfitting due to excessive coefficient shrinkage. As C increased, the mean error decreased steadily, reaching its minimum around C=10. Beyond this point, performance stabilised, with only minor fluctuations in mean error and standard deviation up to C=10000. This suggests that the model had reached a balance between bias and variance.

Unlike Lasso regression, where strong regularisation drives most coefficients exactly to zero, Ridge regression retains all coefficients but shrinks them smoothly. This leads to more stable, less sparse models. Since sparsity is not a factor when deciding the recommended regularisation strength C for Ridge regression as it was for Lasso, the recommended value would be C=10, which corresponds to the lowest mean error (0.0612) with an appropriate standard deviation (0.0116), indicating reliable and consistent performance.

Appendix

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

file = pd.read_csv('week3.csv', header=None)
X = file.iloc[:, 0:2].values  #col1(X[0]->feat1)=x-axis, col2(X[1]->feat2)=y-axis,
y = file.iloc[:, 2].values    #col3(X[2]->target(y))=z-axis
'''----------------------(i)----------------------'''
#----------------------a----------------------
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X[:, 0], X[:, 1], y, marker='o', color='b', label='Training Data')
ax.set_xlabel('Feature 1 (X1)')
ax.set_ylabel('Feature 2 (X2)')
ax.set_zlabel('Target')
ax.set_title('3D Feature Value Scatter Plot')
plt.legend(loc = 'upper right', bbox_to_anchor=(1.20, 1))

plt.show()
#----------------------b(LassoReg)&c----------------------
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Lasso
#generate plynom feats to degree 5
poly = PolynomialFeatures(degree=5, include_bias=False)
#fit plynom feats to data
X_poly = poly.fit_transform(X)
feature_names = poly.get_feature_names_out(['x1', 'x2'])

#c#c#c
    #find training range
#print(X.min(axis=0)) [-1, -1]
#print(X.max(axis=0)) [1, 1]
    #make grid range (a bit over train rng)
x1_range = np.linspace(-1.5, 1.5, 50)
x2_range = np.linspace(-1.5, 1.5, 50)

    #create grid
    #couldn't get nested loops to work properly, used meshgrid() instead
x1_grid, x2_grid = np.meshgrid(x1_range, x2_range)
Xtest = np.c_[x1_grid.ravel(), x2_grid.ravel()]  # shape (2500, 2)
#c#c#c

#regStrenght(C) and alpha values
C_values = [0.1, 1, 10, 100, 1000]
alphas = [1 / (2 * C) for C in C_values]

fig = plt.figure(figsize=(14, 8))
#to record results
results_lasso = []

for i, (C, alpha) in enumerate(zip(C_values, alphas)):
    #train lasso
    model = Lasso(alpha=alpha, max_iter=10000)
    model.fit(X_poly, y)
    results_lasso.append({
        'C': C,
        'Intercept': model.intercept_,
        **dict(zip(feature_names, model.coef_))
    })
    #predict on grid
    Xtest_poly = poly.transform(Xtest)
    y_pred = model.predict(Xtest_poly)
    #reshape to grid
    y_pred_grid = y_pred.reshape(x1_grid.shape)

    #plot
    ax = fig.add_subplot(2, 3, i + 1, projection='3d')
        #pred surface
    ax.plot_surface(x1_grid, x2_grid, y_pred_grid, cmap='viridis', alpha=0.7, label='Lasso
Prediction')
```

```
        #training data
    ax.scatter(X[:, 0], X[:, 1], y, color='r', label='Training data')
        #legend&labels
    ax.set_title(f'Lasso Regression (C={C})')
    ax.set_xlabel('x1'); ax.set_ylabel('x2'); ax.set_zlabel('y')
    if i == len(C_values) - 1:
        ax.legend(loc = 'upper right', bbox_to_anchor=(1.75, 1))

plt.tight_layout()
plt.show()

#convert to csv
df_lasso = pd.DataFrame(results_lasso).round(4)
df_lasso.to_csv('lasso_poly.csv')
pd.set_option('display.max_columns', None)
print(df_lasso)
#----------------------e(RidgeReg)-----------------
from sklearn.linear_model import Ridge
results_ridge = []
fig = plt.figure(figsize=(12, 6))

for i, (C, alpha) in enumerate(zip(C_values, alphas)):
    #train ridge
    model = Ridge(alpha=alpha, max_iter=10000)
    model.fit(X_poly, y)
    results_ridge.append({
        'C': C,
        'Intercept': model.intercept_,
        **dict(zip(feature_names, model.coef_))
    })
    #predict on grid
    Xtest_poly = poly.transform(Xtest)
    y_pred = model.predict(Xtest_poly)
    #reshape to grid
    y_pred_grid = y_pred.reshape(x1_grid.shape)

    #plot
    ax = fig.add_subplot(2, 3, i + 1, projection='3d')
        #pred surface
    ax.plot_surface(x1_grid, x2_grid, y_pred_grid, cmap='viridis', alpha=0.7, label='Ridge
Prediction')
        #training data
    ax.scatter(X[:, 0], X[:, 1], y, color='r', label='Training data')
        #legend&labels
    ax.set_title(f'Ridge Regression (C={C})')
    ax.set_xlabel('x1'); ax.set_ylabel('x2'); ax.set_zlabel('y')
    if i == len(C_values)-1:
        ax.legend(loc='upper right', bbox_to_anchor=(1.75, 1))

plt.tight_layout()
plt.show()

#convert to csv
df_ridge = pd.DataFrame(results_ridge).round(4)
df_ridge.to_csv('ridge_poly.csv')
pd.set_option('display.max_columns', None)
print(df_ridge)

'''------------------------(ii)----------------------'''
#-------------------------a(Lasso)-----------------------
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_squared_error, make_scorer

C_values = [0.01, 0.1, 1, 10, 100, 1000, 10000]
#alpha(lasso) = 1 / (2C)
alphas = [1 / (2 * C) for C in C_values]

results_5FCV = []
mean_errors = []
std_errors = []

for C, alpha in zip(C_values, alphas):
    #train lasso
    model = Lasso(alpha=alpha, max_iter=10000)
```

```
            #since cross_val_score negates values
            scores = cross_val_score(model, X_poly, y, cv=5, scoring='neg_mean_squared_error')
            #convert to positive
            mse_scores = -scores
            #record mse&std.dev
            mean_errors.append(mse_scores.mean())
            std_errors.append(mse_scores.std())
            results_5FCV.append({
                "C": C,
                "Alpha": alpha,
                "Mean_MSE": mse_scores.mean(),
                "Std_MSE": mse_scores.std()
            })

        #convert to csv
        cv_Lasso = pd.DataFrame(results_5FCV)
        cv_Lasso.to_csv('5FCV_Lasso.csv')
        print(cv_Lasso)

        #plot
        plt.figure(figsize=(8,5))
            #for std.dev of mse
        plt.errorbar(C_values, mean_errors, yerr=std_errors, fmt='-o', capsize=4)
            #labels
        plt.xscale('log')
        plt.xlabel('C (Regularisation Strength)')
        plt.ylabel('Mean Squared Error ± Std.Deviation')
        plt.title('5-fold Cross-Validation Error - Lasso Regression')

        plt.grid(True)
        plt.show()

        #------------------------c(Ridge)-----------------------
        rdgResults_5FCV = []
        rdgMean_errors = []
        rdgStd_errors = []

        #C vals already init(^)
        #alpha(ridge) = 1 / (2C)
        alphas = [1 / C for C in C_values]
        for C, alpha in zip(C_values, alphas):
            #train ridge
            model = Ridge(alpha=alpha)
            #since cross_val_score negates values
            scores = cross_val_score(model, X_poly, y, cv=5, scoring='neg_mean_squared_error')
            #convert to positive
            mse_scores = -scores
            #record mse&std.dev
            rdgMean_errors.append(mse_scores.mean())
            rdgStd_errors.append(mse_scores.std())
            rdgResults_5FCV.append({
                "C": C,
                "Alpha": alpha,
                "Mean_MSE": mse_scores.mean(),
                "Std_MSE": mse_scores.std()
            })
        #convert to csv
        cv_Ridge = pd.DataFrame(rdgResults_5FCV)
        cv_Ridge.to_csv('5FCV_Ridge.csv')
        print(cv_Ridge)

        #plot
        plt.figure(figsize=(8,5))
            #for std.dev of mse
        plt.errorbar(C_values, rdgMean_errors, yerr=rdgStd_errors, fmt='-o', capsize=4)
            #labels
        plt.xscale('log')
        plt.xlabel('C (Regularisation Strength)')
        plt.ylabel('Mean Squared Error ± Std.Deviation')
        plt.title('5-fold Cross-Validation Error - Ridge Regression')

        plt.grid(True)
plt.show()
```